

exámenes-ec.pdf



kamorebi



Estructura de Computadores



2º Grado en Ingeniería Informática



**Facultad de Informática
Universidad Complutense de Madrid**



ESTRUCTURA DE COMPUTADORES

Examen - 11 de junio de 2021

Nombre _____ DNI _____

1. (1 punto)

- (0,5 pts) Expresar en punto flotante de precisión simple, según el estándar IEEE 754, el número $(10,25)_{10}$.
- (0,5 pts) Expresar en decimal el número $0xBE60_0000$, representado en punto flotante de precisión simple según el estándar IEEE 754.

2. (3 pts) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de ejecución.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	4	Sí
FP MUL	1	5	Sí
INT ALU	1	1	No

- (2 pts) Completar el diagrama instrucción-tiempo para los primeros 24 ciclos del siguiente fragmento de código, indicando claramente los cortocircuitos realizados, las paradas y sus causas.

```
loop:  ADDD    F6, F5, F2
      ADDD    F2, F1, F3
      SUBI    R5, R5, #4
      LD      F4, 0(R5)
      ADDD    F6, F6, F2
      MUL    F5, F6, F4
      BNE    R5, R0, loop
      SD     F5, 256(R5)
```

- (1 pto) Calcular el CPI de la ejecución completa del código, siendo $R0=0$ y $R5=32$ al comienzo de la ejecución.

3. (3 pts) En un computador con caches separadas de datos e instrucciones, de 128 bytes cada una, emplazamiento directo y bloques de 16 bytes, se quiere ejecutar el siguiente código C:

```
#define N 64;
int A[N][N]; /* Un entero ocupa 32 bits
int B[N];
int C[N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        temp=temp + B[j] * A[i][j];
    C[i] =temp;
```

Ignorando los accesos a la variable **temp** y suponiendo que los tres arrays se colocan en memoria de modo consecutivo a partir de la dirección $0x0B00_0000$.

- a) (1 pto) Calcular el número de fallos de bloque en la cache de datos si se usa escritura directa sin asignación en escritura
- b) (1 pto) Si añadimos una memoria cache de segundo nivel unificada con bloques de 32 bytes, asociativa por conjuntos con dos vías y tamaño 256 bytes. Calcular el número de fallos de bloque por datos en cada nivel de cache si en ambos niveles de cache se usa escritura directa con asignación en escritura.
- c) (1 pto) Si la latencia de acceso a la cache L2 es 10 ciclos y la tasa de transferencia entre L1 y L2 es de 16 bytes por ciclo, y la latencia de acceso a memoria principal desde L2 es de 50 ciclos y la tasa de transferencia es de 8 bytes por ciclo, calcular la penalización que se ha producido por los fallos de acceso a memoria cache. Suponer que los tiempos de escritura se solapan con los de lectura.
4. (3 ptos) En una máquina con una memoria principal de 4 KB el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 1KB y un algoritmo LRU de reemplazo de páginas. Siendo el tamaño de la memoria virtual 8 KB, se pide:
- a) (1 pto) Indica cuál es el contenido de la tabla de páginas y de una TLB de 2 entradas (LRU), tras el acceso a las siguientes direcciones virtuales de memoria: 0x04C0, 0x1680, 0x09C4, 0x06A0 y 0x0CCC, suponiendo que anteriormente la memoria principal estaba vacía y que las páginas físicas se han asignado en orden creciente a partir de la página 0. Utiliza el valor LRU = 0 para la última página referenciada.
- b) (1 pto) En un momento dado el contenido de la tabla de páginas es el que se muestra a continuación. Indica cómo evoluciona la tabla de páginas tras realizar las siguientes referencias a memoria: 0x12FC, 0x0540, 0x18C0, 0x0380, 0x1C44.
Si a continuación se referencia de nuevo la dirección 0x12FC, indica cuáles son sus direcciones físicas cada vez que se referenció. ¿Son iguales? Razona la respuesta.

	Válido	PF	LRU
0	0		
1	1	0	2
2	0		
3	1	3	0
4	1	2	3
5	0		
6	0		
7	1	1	1

- c) (0,5 ptos) Se añade una memoria cache de 128 bytes con bloques de 32 bytes, virtualmente accedida, completamente asociativa y con algoritmo de reemplazo FIFO. En un momento dado el contenido de la TLB y de la memoria cache es el que se muestra a continuación. Mostrar los rangos, que se puedan con la información disponible, de direcciones físicas y virtuales de los bloques ubicados en la memoria cache.

Bloque MC	Etiqueta	FIFO
0	0x45	2
1	0x8A	0
2	0xFF	1
3	0x68	3

TLB		
PV	PF	LRU
2	2	0
4	3	1

- d) (0,5 ptos) Partiendo del contenido de la memoria cache y de la TLB del apartado anterior, indica detalladamente y en orden, qué sucede cuando se referencia la dirección de memoria 0x1280.
El valor FIFO = 0 indica el último bloque que se ha traído a la memoria cache.



ESTRUCTURA DE COMPUTADORES

Examen - 14 de julio de 2021

Nombre _____ DNI _____

1. (1 punto)

- (0,5 ptos) Expresar en punto flotante de precisión simple, según el estándar IEEE 754, el número hexadecimal: +8452.
- (0,5 ptos) Expresar en decimal el número 0xBB50_0000, representado en punto flotante de precisión simple según el estándar IEEE 754.

2. (3 ptos) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos o cortocircuito desde los registros del pipeline.
- Los saltos se resuelven en la etapa ID y existe cortocircuito combinacional.
- La detección de todo tipo de riesgos (estructurales y LDE) y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos EDE se resuelven mediante paradas hasta que la instrucción ya lanzada entre en la etapa de memoria (en ese momento la segunda instrucción saldría de su etapa ID).
- Se implementan saltos retardados con un delay-slot de 1 ciclo.

Las unidades funcionales de las que dispone el procesador son las siguientes:

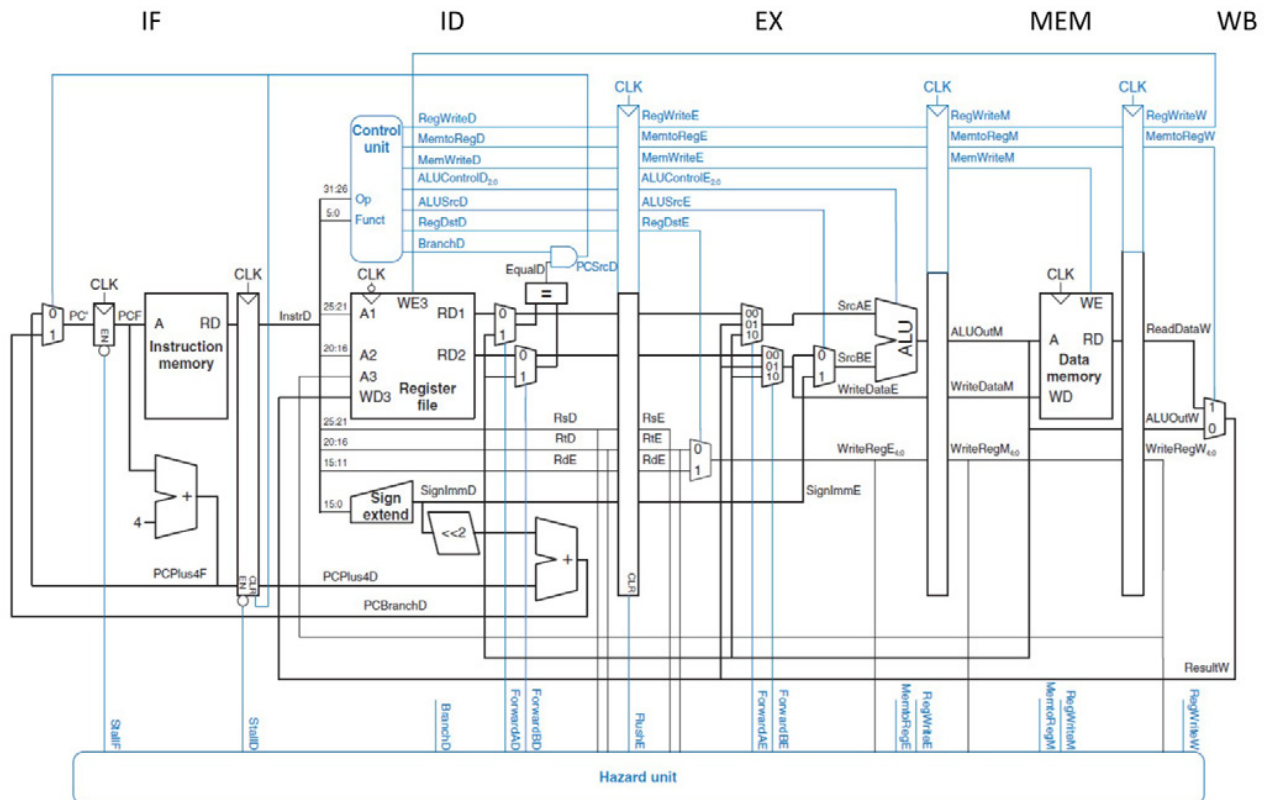
UF	Cantidad	Latencia	Segmentación
FP ADD	1	2	Sí
FP MUL	2	3	No
FP DIV	1	5	No
INT ALU	1	1	No

- a) (2 ptos) Completar el diagrama instrucción-tiempo correspondiente a los 26 primeros ciclos de ejecución del siguiente fragmento de código, indicando claramente los cortocircuitos realizados, las paradas y sus causas. Suponer que R1=0 y R2=80 al comienzo de la ejecución.

```
LD F1, 0(R1)
loop: MULF F2,F1,F0
      MULF F0,F1,F3
      MULF F3,F1,F4
      SDF3,0(R1)
      ADDI R1,R1,8
      BNE R1,R2,loop
```

- b) (1 pto) Suponiendo que el siguiente código se almacena a partir de la dirección de memoria 0x0C000000, indique el contenido de los registros y el valor de las señales de control de la tabla, en el ciclo 7.

	1	2	3	4	5	6	7	8	9	10	11	12
LW R1,0(R2)	IF	ID	EX	M	WB							
ADD R1,R1,R3		IF	IDp	ID	EX	M	WB					
SW R1, 0(R2)			IFp	IF	ID	EX	M	WB				
ADD R2,R2,R5					IF	ID	EX	M	WB			
BEQ R2,R4,loop						IF	IDp	ID	EX	M	WB	
ADDI R3,R3,1							IFp	IF	ID	EX	M	WB



PCF		BranchD		WriteRegE	
StallF		FlushE		RegWriteM	
StallID		RegWriteE		MemWriteM	
RsD		MemWriteE		MemtoRegM	
RtD		RsE		RegWriteW	
RegWriteD		RtE		WriteRegW	
MemWriteD		RdE		MemtoRegW	

3. (3 pts) En un computador con caches separadas de datos e instrucciones de 128 bytes asociativas por conjuntos con dos vías, bloques de 16 bytes, política LRU (el bloque más reciente tiene LRU=0) y escritura directa con asignación en escritura, se quiere ejecutar el siguiente código C:

```

#define N 64;
int A[N];      /* Un entero ocupa 32 bits
int B[N];
int C[N];
int D;

D = 0;
for (j = 0; j < N-1; j += 2) {
    D = D + A[j] + B[j] + C[j];
    D = D + C[j+1] + B[j+1] + A[j+1];
}

```

Sabiendo que las variables j y D están en registros del procesador, que los arrays A, B y C se colocan en memoria de modo consecutivo a partir de la dirección 0x0B00_0000 y asumiendo que los arrays se acceden en el orden en el que están en el código (para la primera sentencia, primero la componente de A, después B y finalmente C; para la segunda sentencia, primero la componente de C, después B y finalmente A), se pide:

- (1,25 pto) Calcular el número de fallos de bloque de cada tipo en la cache de datos.
- (1 pto) Si se modifica el tamaño de bloque a 32 bytes sin cambiar el tamaño de la cache, razonar si cambia el número de fallos de cache. De ser así, calcular el nuevo valor.
- (0,75 pto) Reorganizar el código original para reducir el número de fallos para la configuración original de la cache sin cambiar el tamaño ni la ubicación de los arrays.

4. (3 pto) Dado un computador con memoria virtual paginada con direcciones virtuales de 24 bits, páginas de 64KB y memoria física de 4MB. Los contenidos iniciales de la tabla de páginas de un proceso son:

	Válido	PF	LRU
1	1	22	0
7	1	11	1
9	1	7	2
12	1	5	3

Solo se muestran las filas de la tabla de páginas con bit de válido a 1.

La página más recientemente usada tiene LRU=0.

Los números de página virtual y física están en decimal.

- (1,25 pto) Para la siguiente secuencia de accesos, decir si son acierto o fallo y cómo se modifica la tabla de páginas: 0x078010, 0x059500, 0x07A100, 0x0CA360, 0x07C000. Suponer que sólo se dispone de 4 marcos de página en la memoria principal para las páginas de este proceso.
- (0,75 pto) Se añade un TLB completamente asociativo, con algoritmo de reemplazo LRU (LRU=0 indica la entrada más reciente). Si en un momento dado el contenido del TLB es el siguiente, indicar el rango de direcciones físicas y virtuales que se han accedido más recientemente.

TLB		
PV	PF	LRU
22	12	0
47	03	1

- (1 pto) Se añade una memoria cache asociativa por conjuntos de dos vías virtualmente accedida y físicamente marcada de 8 marcos de tamaño 16KB cada uno y algoritmo de reemplazamiento FIFO, que inicialmente está vacía. Repetir el apartado a) incluyendo los accesos a cache.



ESTRUCTURA DE COMPUTADORES
Examen Convocatoria Extraordinaria. Junio de 2019

Nombre _____ DNI _____
Apellidos _____ Grupo _____

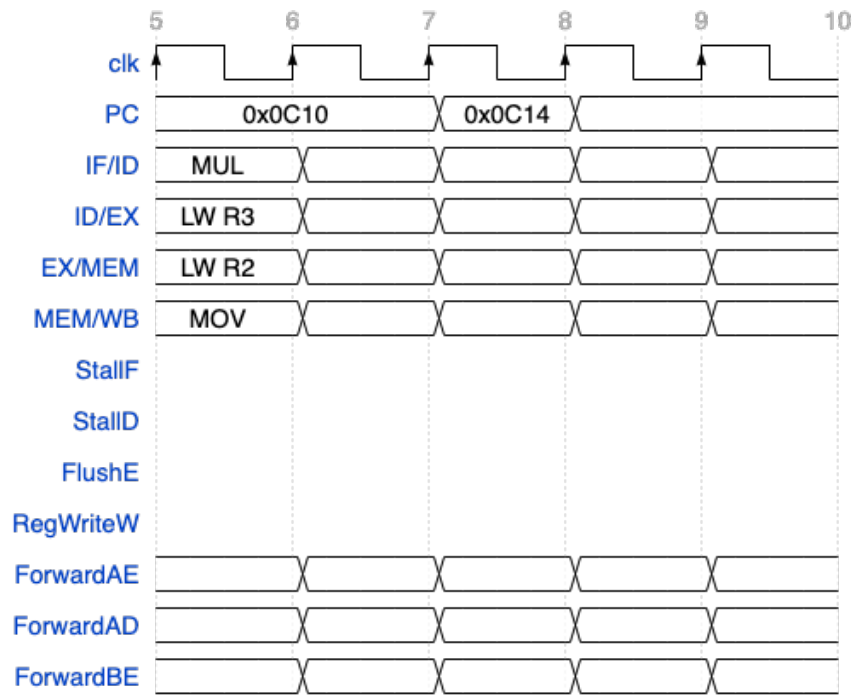
1. (4 puntos) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en decodificación.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	2	Sí
FP MUL	1	3	Sí
INT ALU	1	1	No

a. (1 punto) Dado el siguiente diagrama instrucción-tiempo, complete el cronograma. Supóngase que la unidad de detección de riesgos es combinacional. Indíquese también sobre el cronograma los flancos en los que se produce la escritura del banco de registros.

Addr.	Ciclo	1	2	3	4	5	6	7	8	9	10	11	12	13
0x0C04	bucle: LW R2, 0(R1)		IF	ID	E	M	WB							
0x0C08	LW R3, 128(R1)			IF	ID	E	M	WB						
0x0C0C	MUL R4, R3, R2				IF	IDp	ID	E	M	WB				
0x0C10	ADD R6, R6, R4					IFp	IF	ID	E	M	WB			
0x0C14	SUBI R1, R1, #4							IF	ID	E	M	WB		
0x0C18	BNEZ R1, bucle							IF	IDp	ID	E	M	WB	
0x0C1C	SW R6, 256(R1)								IFp	IF	ID	---	---	



- b. (2 puntos) Represente el diagrama instrucción-tiempo para la primera iteración del código que se muestra a continuación, indicando los cortocircuitos realizados, las paradas y sus causas. Inicialmente R1=64 y R2=1.

```

MOV    R2, #1
loop:  BEQ    R0, R2, mult
      LW     F0, 0(R1)
      LW     F1, 128(R1)
      MULS   F4, F0, F1
      ADDS   F5, F5, F4
      MOV    R2, #0
mult:  LW     F0, 0(R1)
      ADDS   F5, F5, F0
      SUBI   R1, R1, #4
      BNEZ   R0, R1, loop
      SW     F5, 256(R1)
      MOV    R2, #1

```

- c. (1 punto) Aumentar la frecuencia de reloj del procesador de 1 GHz a 1.5 GHz impone usar una caché de datos segmentada lo que requiere segmentar las etapas de memoria y de decodificación en dos etapas cada una de ellas, incrementando el total de etapas del procesador hasta 7. En este caso, la detección de riesgos LDE, la generación de paradas y el cortocircuito a la etapa ID para las instrucciones de salto se resuelven en la segunda etapa de decodificación. Determinar el speedup de la ejecución de todas las iteraciones del código en esta implementación tomando la implementación inicial (analizada en el apartado b.) como caso base.

2. **(3 puntos)** Un pequeño microcontrolador basado en ARM dispone de una memoria principal de 512KB con cachés separadas de datos e instrucciones de 128B cada una, de emplazamiento directo y bloques de 16B. La caché de datos tiene asignación en escritura (write allocate). En dicho sistema se quiere evaluar el rendimiento del siguiente fragmento de un programa:

```
for (i=0; i < 6; i++)  
    for (j=1; j<7; j++)  
        A[i][j] = (B[i][j]+ B[i][j+1])/2;
```

Sabiendo que A y B son matrices de 6x8 de enteros (un entero son 4 bytes), que están ubicadas en memoria en las direcciones B: 0x15000 y A: 0x20310 y que se almacenan por filas. Responda razonadamente a las siguientes cuestiones:

- a. **(1.5 puntos)** ¿Cuál es la tasa de fallos de la caché de datos al ejecutar el código?
- b. **(1.5 puntos)** Si se añade al microcontrolador una cache de víctimas de 32B totalmente asociativa y con algoritmo de reemplazamiento LRU repetir el apartado a, detallando qué bloques se encuentran en la caché de datos y en la de víctimas al finalizar la primera iteración del bucle externo (i ==0).
3. **(3 puntos)** En una máquina con una memoria DRAM de 256MB y un procesador de 32 bits (direcciones virtuales de 32 bits) el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 4KB y con algoritmo de reemplazamiento LRU. En dicha máquina se ejecuta una aplicación que accede a distintas estructuras de datos, cada una de las cuales ocupa 1KB. El proceso dispone de los marcos de página 6, 7, 8 y 9 para almacenar el código y los datos (se asignan en ese orden, esto es, la primera página que necesite el proceso se alojará en el marco 6). Se dispone de una TLB de dos entradas totalmente asociativa.

Las direcciones virtuales iniciales de cada una de las estructuras de datos son:

Nombre E. D.	A	B	C	D	E	F
D. V. inicial	0x00128000	0x00941600	0x00002000	0x005110A0	0x00300100	0x00041000

El código empieza en la dirección virtual 0x000C0000 y ocupa 3500B. Tener en cuenta que cada vez que se ejecuta una instrucción se accede a la página de código que almacena esta instrucción.

Se pide:

- a. **(1.5 puntos)** Mostrar el estado final de la tabla de páginas y la TLB cuando la aplicación se ejecuta y accede a las estructuras de datos en el siguiente orden: A, B, A, C, D, A, E, B, A.
- b. **(1.0 puntos)** Si a continuación el proceso intenta acceder a F, indique las estructuras que se consultarían, el orden en que se haría y si habría fallo o acierto en cada caso. En caso de que se produjeran fallos indique cómo quedaría cada una de las estructuras tras resolver el fallo.
- c. **(0.5 puntos)** Si la memoria cache de datos fuera virtualmente accedida físicamente marcada, ¿cuál sería su tamaño máximo para asociatividad 2?



ESTRUCTURA DE COMPUTADORES
Examen Convocatoria Ordinaria. 3 de Junio de 2019

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1. **(4 puntos)** Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en decodificación.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	2	Sí
FP MUL	1	4	Sí
INT ALU	1	1	No

El siguiente código calcula el producto escalar de dos vectores, realizando dos productos por iteración:

```
MOV R1, #64
bucle: LS F0, 0(R1)
      LS F2, 128(R1)
      MULS F4, F0, F2
      ADDS F6, F6, F4
      SUBI R1, R1, #4
      LS F0, 0(R1)
      LS F2, 128(R1)
      MULS F4, F0, F2
      ADDS F6, F6, F4
      SUBI R1, R1, #4
      BNEZ R1, bucle
      SS F6, 256(R1)
```

- (1.5 puntos)** Represente el diagrama instrucción-tiempo para la primera iteración, indicando los cortocircuitos realizados, las paradas y sus causas.
- (1 punto)** Proponga una optimización del código que disminuya el número de ciclos de ejecución. Realice una estimación del ahorro conseguido.
- (1.5 puntos)** Considere ahora la ejecución del código de la siguiente tabla. Indique y justifique el valor de las señales StallF, FlushE, A1, RsE, RdE, ForwardBE y ForwardAE en los ciclos 5, 6, 7 y 8.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
mov r1, #64	IF	ID	E	M	WB									
bucle: lw r2, 0(r1)		IF	ID	E	M	WB								
lw r3, 128(r1)			IF	ID	E	M	WB							
mul r4, r3, r2				IF	IDp	ID	E	M	WB					
add r6, r6, r4					IFp	IF	ID	E	M	WB				
subi r1, r1, #4							IF	ID	E	M	WB			
bnez r1, bucle								IF	IDp	ID	E	M	WB	
sw r6, 256(r1)									IFp	IF	ID	E	M	WB

2. **(3 puntos)** Un pequeño microcontrolador basado en ARM dispone de una memoria principal de 64KB con cachés separadas de datos e instrucciones de 64B cada una, de emplazamiento directo y bloques de 16B. La caché de datos tiene asignación en escritura (write allocate) y actualización en post-escritura (write-back). En dicho sistema se quiere evaluar el rendimiento del siguiente fragmento de un programa:

```
for (i=0; i < 40; i++)
    A[i] += B[i];
```

que al compilarse resulta en el siguiente código ensamblador, ubicado en la dirección 0xC020:

```
    ldr r5, =A
    ldr r6, =B
    mov r4, #40
L1:  ldr r0, [r5]
     ldr r1, [r6], #4
     add r0, r1, r0
     str r0, [r5], #4
     subs r4, r4, #1
     bgt L1
```

Sabiendo que A y B son arrays de 40 enteros (un entero son 4 bytes), que están ubicados en memoria uno a continuación del otro a partir de la dirección 0x1000, y que el contenido de los directorios al comienzo de la ejecución de este fragmento de código es el siguiente:

IL1		
	v	etiqueta
0	1	0x301
1	1	0x301
2	1	0x300
3	1	0x300

DL1		
	v	etiqueta
0	1	0x042
1	1	0x042
2	1	0x044
3	1	0x044

Responda razonadamente a las siguientes cuestiones:

- (1.5 puntos)** ¿Cuál es la tasa de fallos de ambas cachés al ejecutar el código?
- (0.75 puntos)** Una modificación algorítmica requiere que los arrays pasen a ser de 48 enteros. Indique cómo afectaría esto al rendimiento del código. Indique la nueva tasa de fallos de ambas cachés.
- (0.75 puntos)** Proponga alguna transformación de código que mejore el rendimiento de la caché con arrays de 48 enteros, indicando la tasa de fallos que obtiene en ese caso.

3. **(3 puntos)** En una máquina con una memoria DRAM de 256MB y un procesador de 32 bits (direcciones virtuales de 32 bits) el sistema operativo gestiona la memoria virtual por paginación, utilizando páginas de 4KB. En dicha máquina se ejecuta un servidor que ofrece consultas a una base de datos. Para ser más rápido, el servidor mantiene mapeados en su espacio virtual de memoria los 8 ficheros de la base de datos consultados más recientemente. Cada fichero ocupa 1KB. En un momento dado los ficheros mapeados en memoria virtual son los indicados por la tabla de la izquierda. La tabla de la derecha nos muestra las entradas más relevantes de la tabla de páginas del proceso.

Ubicación de ficheros en M.V.

Fichero	D. Virtual
Fch 1	0x000A0000
Fch 3	0x00020000
Fch 2	0x000A0C00
Fch 4	0x00020400
Fch 8	0x000A0800
Fch 12	0x00060C00
Fch 13	0x000A0400
Fch 20	0x00060800

Tabla de Páginas

v	P. Física
0	...
1	0x0021
0	...
0	0x0003
1	0x0040
0	...
1	0x0001
0	...

Se pide:

- (1 punto)** Indique para cada fichero ubicado en memoria virtual si se encuentra en memoria DRAM (física) o no, y en caso afirmativo el rango de direcciones físicas que ocupa.
- (1 punto)** El sistema dispone además de una caché física de 4 bloques de 64 bytes por bloque, de emplazamiento directo y una TLB de dos entradas completamente asociativa. En un momento dado los contenidos del directorio de la caché y la TLB son los siguientes (LRU = 0 es el que se ha accedido más recientemente):

Directorio

	v	Etiqueta
0	1	0x00014
1	1	0x00014
2	1	0x00215
3	1	0x00215

TLB

P. Virtual	P. Física	LRU
0x000A0	0x0001	0
0x00020	0x0021	1

Indique cuál es el fichero que ha sido accedido más recientemente, justificando la respuesta.

- (1 punto)** Si a continuación el proceso intenta acceder al segundo bloque que forma parte de Fch 12, indique las estructuras que se consultarían, el orden en que se haría y si habría fallo o acierto en cada caso. En caso de que se produjeran fallos indique cómo quedaría cada una de las estructuras tras resolver el fallo. En caso de fallo de página asumir que se usa como página física libre la 0x0002.



ESTRUCTURA DE COMPUTADORES

Examen Final - Junio de 2018

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un procesador segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos. El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID para las instrucciones de salto, que se resuelven en dicha etapa.
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- No se implementan saltos retardados, sino que se busca siempre la siguiente instrucción al salto y se cancela su ejecución si el salto es tomado.
- Las unidades funcionales de las que dispone el procesador son las siguientes:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	3	Sí
FP MUL	1	5	Sí
Int ALU	1	1	No

El siguiente código calcula los elementos del vector `double C[128]` a partir de los elementos de los vectores `double A[128]` y `double B[128]` (recuerda que un elemento en punto flotante de doble precisión ocupa 8 bytes). Los vectores se encuentran almacenados en memoria de forma consecutiva (A-B-C) y la dirección de comienzo del vector A es `0x80001000`.

LOOP:

```
LD      F2, 0(R6)
AND     R2, R4, R9
BEQ     R2, R0, ELSE
IF:     ADDD  F6, F2, F2
        MULD  F6, F8, F6
        BEQ   R0, R0, END
ELSE:   LD    F4, 1024(R6)
        MULD  F6, F2, F4
        ADD   R10, R10, R2
        ADDD  F10, F2, F4
END:    SD    F6, 2048(R6)
ADD     R6, R6, R3
ADD     R4, R4, R9
BNE     R4, R5, LOOP
SUB     R4, R4, R4
```

R6=0x80001000

R3=8

R5=128

R4=0

R9=1

R10=0

R7=-1

F8=5.5

R0 =0 siempre en la
arquitectura MIPS

El contenido inicial de los registros es el siguiente:

- (2 puntos) Representa el diagrama instrucción-tiempo para la primera y segunda iteración e indica los cortocircuitos realizados. Indica claramente las paradas y sus causas.
- (1 punto) Calcula el CPI de la ejecución completa del código.
- (1 punto) Si el procesador utilizara saltos retardados, ¿qué instrucciones llevarías al *delay-slot* de cada uno de los saltos y qué efectos tendría?

2.- (3 puntos) En un sistema con procesador ARM y una memoria de 64KB se quiere analizar el comportamiento del siguiente fragmento de un programa.

```

.global start
.equ N, 12

.data
A: .word N valores separados por comas
B: .word N valores separados por comas

.bss
C: .space 4*N

.text
....
mov R3, #N
mov R4, #0
mov R5, #0
mov R2, #0
for: cmp R2, R3
     bge fin
     ldr R4, [R0, R2, LSL#2]
     ldr R5, [R1, R2, LSL#2]
     add R4, R4, R5
     str R4, [R7, R2, LSL#2]
     add R2, R2, #1
     b for

```

El programa se enlaza ubicando la sección .data a partir de la **dirección 0x0090** y la sección .bss a continuación de la sección .data, y que el fragmento de la sección .text en la que se encuentra la primera instrucción (mov R3, #N) se ubica en la **dirección 0x0140**

R0, R1 y R7 contienen las direcciones de A, B y C respectivamente

Responde de forma razonada a las siguientes preguntas:

- (0.5 puntos) Obtener el rango de direcciones que ocupa el fragmento de programa y cada uno de los arrays. ¿Cuántos bloques de memoria principal ocupan? ¿Qué bloques son?
- (1.5 puntos) El sistema dispone de caches separadas de datos (D\$) e instrucciones (I\$) de 32 bytes cada una, ambas con política de emplazamiento directo, y políticas de escritura directa (write through) con asignación en escritura (write allocate), y tamaño de bloque de 16 bytes. Obtener los fallos que se producirán en la cache de datos (D\$) y en la cache de instrucciones (I\$) al ejecutar el fragmento del programa. Indicar, además del número total de fallos, para qué bloques de memoria principal se producen los fallos.
- (1 punto) Ahora añadimos al sistema de memoria una cache de segundo nivel unificada para datos e instrucciones (L2\$) de 128 bytes, asociativa por conjuntos de 2 vías, con reemplazamiento LRU y políticas de escritura directa (write through) con asignación en escritura (write allocate). Indicar el número de fallos que se producen en L2 tras ejecutar el fragmento del programa. (El tamaño de bloque es también de 16 bytes).

3.- (3 puntos) En una empresa dedicada al análisis de datos tienen que utilizar un algoritmo que requiere sumar todos los elementos de cada columna de una matriz de 512x512 elementos. El código C de dicho algoritmo es el siguiente:

```

for (j = 0; j < 512; j++)
    for (i = 0; i < 512; i++)
        red[j] = red[j] + A[i][j];

```

Sabiendo que:

- En C los elementos de una matriz se emplazan en memoria consecutivos por filas
- El procesador es de 32 bits (direcciones virtuales de 32 bits).
- El array de enteros A[512][512] se ubica en el mapa virtual de memoria del proceso a partir de la dirección **0x00458000**, y el array de enteros red[512] se ubica justo a continuación de A.
- El sistema operativo (SO) maneja páginas de 4KB con reemplazo **LRU**.

se pide:

- (0.75 puntos) Indicar qué páginas de memoria virtual ocupan los arrays A y red, indicando qué elementos de cada array están en cada página.
- (1.25 puntos) El SO le asigna al proceso 8 marcos de página (lo que sería equivalente a decir que, desde el punto de vista de este proceso, la memoria física tiene 8 páginas). Sabiendo que 2 de esos 8 marcos están ocupados por la página de código y la página de pila, accedidos constantemente y que por tanto nunca son elegidos para el reemplazo, y que los 6 restantes no contienen inicialmente ninguna página de A ni de red; indicar razonadamente cuántos fallos de página se producirán al ejecutar el código y el número de accesos a A y red que no suponen fallo de página.
- (1 punto) Razone si es posible o no reducir significativamente los fallos de página del proceso con alguna modificación en su código, indicando en su caso qué transformación se haría y cuantificando la mejora esperada.



ESTRUCTURA DE COMPUTADORES
Examen Final – Septiembre 2018

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un DLX segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos (tiene implementado cortocircuito).
- El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID (para instrucciones de salto).
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- Los saltos se resuelven en la etapa de decodificación y se implementan saltos retardados con un “delay-slot” de 1 ciclo.
- Las unidades funcionales del procesador son:

UF	Cantidad	Latencia	Segmentación
FP ADD	2	3	No
FP MUL	1	5	Sí
Int ALU	1	1	No

El siguiente fragmento de código se ejecuta sobre dicho procesador:

```
Bucle:  LD F1, 0(R3)
        LD F2, 0(R4)
        LD F3, 0(R5)
        LD F4, 0(R6)
        AND R7, R2, R1
        BNE R7, R0, Salto
        NOP
        ADDD F1, F1, F2
        ADDD F3, F3, F4
        MULF F1, F1, F3
Salto:  SD F1, 0(R3)
        ADDI R3, R3, 8
        ADDI R4, R4, 8
        ADDI R5, R5, 8
        ADDI R6, R6, 8
        SUBI R2, R2, 1
        BNE R2, R0, Bucle
        NOP
```

Suponiendo que los siguientes registros están inicializados al valor indicado a continuación:

- R0 = 0
 - R1 = 1
 - R2 = 100
- a) (2 puntos) Representar el diagrama instrucción-tiempo para la primera iteración del bucle indicando claramente los cortocircuitos realizados, las paradas y sus causas.
- b) (0.75 puntos) Calcular el CPI de la ejecución completa del código dado.
- c) (0.75 puntos) Reordenar el código para reducir el número de ciclos que tarda en ejecutarse, intentando evitar la ejecución de operaciones innecesarias en algunas iteraciones.
- d) (0.5 puntos) ¿Podrían sustituirse las operaciones NOP del código por otras? ¿Por cuáles?

2.- (3 puntos) Sea un computador con un sistema de memoria con las siguientes características:

- Una memoria principal 64KB
- Una cache de datos de 128 Bytes, con emplazamiento directo, sin asignación en escritura (non write-allocate) y política de actualización post-escritura (write-back)
- Tamaño de bloque de 16 bytes

Se quiere ejecutar el siguiente código C:

```
int mask[4]; // un entero ocupa 4 bytes
int filter[12];
int M[16][16];

int suma = 0;
for (i=0; i<16; i+=2)
    for (j=0; j<16; j+=4)
        suma += mask[0]*M[i][j] + mask[1]*M[i][j+1];
```

***** IMPORTANTE: el incremento de las variables i y j en el bucle NO es de una unidad. ******

Asumiendo que el array **mask** se ubica en la dirección **0x0C00**, que **filter** y **M** se almacenan consecutivamente a continuación de **mask**, que en la traducción a ensamblador del código anterior las variables i , j , **suma** y las direcciones de comienzo de **mask**, y **M** se ubican en registros, y que el orden relativo de accesos en cada iteración es **mask – M – mask – M**, se pide:

- a) (0.75 puntos) Calcular el número de fallos de cache que se producen en la ejecución de ese programa. ¿Cuántos bloques de cache se utilizan durante su ejecución? Justifique su respuesta.
- b) (0.75 puntos) Se considera introducir una cache de víctimas de 4 bloques y totalmente asociativa. Discuta si, para este ejemplo, se reduciría la tasa de fallos y, si es así, haga una estimación de dicha reducción.
- c) (0.75 puntos) Se considera ahora implementar un mecanismo simple de prebúsqueda bajo fallo (en caso de fallo se trae el bloque que lo provoca y el siguiente a la cache). ¿Se reduciría, para este ejemplo, el número de fallos? Si es así, haga una estimación de dicha reducción
- d) (0.75 puntos) Tras trabajar en otras partes del código, se decide ampliar el array **filter** de modo que pase a tener 16 elementos en lugar de 12. ¿Afectaría en algo al número total de fallos? ¿Cuántos se producirían tras esa modificación de código?

3.- (3 puntos) Una máquina dispone de un procesador de 32 bits (direcciones virtuales de 32 bits) y una memoria DRAM de 256MB (memoria física). El sistema operativo gestiona la memoria virtual con paginación, utilizando páginas de 4KB. Asimismo, se dispone de una cache de acceso directo, físicamente accedida, con 16 bloques de 16 bytes cada uno. Un proceso que ejecuta en dicho sistema tiene una matriz A de 64x64 enteros (1 entero ocupa 4 bytes) ubicada a partir de la dirección 0x0C000400 de su mapa virtual de memoria. La tabla de páginas de dicho proceso es la siguiente

	V	Nº pág. física
0x00000	0	
...	...	
0x0BFFF	1	0x0004
0x0C000	1	0x0010
0x0C001	1	0x0002
0x0C002	0	
0x0C003	0	
0x0C004	1	0x0006
0x0C005	1	0x000A
0x0C006	0	
...	...	
0x0C100	1	0x1000
0x0C101	1	0x1001
...	...	
0x7FFFF	1	0x4000
...	...	

- a. (0.75 puntos) Indicar los formatos de las direcciones virtual y física, esta última también desde el punto de vista de la cache.
- b. (0.75 puntos) Indica las páginas virtuales en las que se almacena la matriz A y qué elementos del array están en cada página. Indica cuáles de ellas están en memoria y cuáles no.
- c. (0.75 puntos) En un momento de la ejecución, los únicos bloques válidos de la cache son los que contienen los elementos A[2][10], A[14][2] y A[14][7]. Indica el contenido de la cache, incluyendo etiquetas, bit V y elementos de A de cada bloque.
- d. (0.75 puntos) Indica todo lo que sucedería (y en qué orden) si la CPU intenta acceder a memoria en la dirección 0x0C002300. Asume que el contenido de la cache es el indicado en el apartado anterior. En caso de fallo de página, el algoritmo de reemplazamiento elegirá aleatoriamente uno de los tres marcos actualmente usados por la matriz A. ¿Hay fallo de página? ¿Y de cache? Si hay fallo de cache, ¿qué datos nuevos se alojarán y con qué etiqueta? ¿En qué marco de cache?

	ESTRUCTURA DE COMPUTADORES Examen Final Junio - 13 de Junio de 2014
	Nombre _____ DNI _____ Apellidos _____ Grupo _____

- (1,5 ptos)** Se dispone de un disco duro con un ancho de banda de 10MB/s conectado a memoria principal a través de un bus síncrono de frecuencia 500MHz y ancho de datos de 32 bits. El disco duro dispone de un buffer de hasta 1KB donde puede almacenar los datos antes de enviarlos por el bus. La gestión de la transferencia de datos se realiza a través de DMA en modo ráfaga.
 - Suponiendo que los datos no se envían por el bus hasta que no se ha llenado el buffer. ¿Cuánto tiempo tarda el proceso de DMA en enviar 1KB? ¿Cuánto tiempo permanece la CPU inactiva?
 - A ese mismo bus se conecta otro dispositivo con un ancho de banda de 12,5 MB/s que toma el control del bus cada vez que necesita enviar 32 bits. Durante la operación de DMA de 1KB, ¿cuántas veces (como máximo) colisiona ésta con el otro dispositivo de entrada/salida por el control del bus?
 - Para que no se produzca la colisión, ¿cuál debería ser el tamaño máximo de datos enviado en modo ráfaga?
- (1,5 ptos)** Sea un procesador segmentado con cinco etapas como el visto en clase. Ejecutamos en este procesador una aplicación con las siguientes características:
 - El 20 % de las instrucciones son saltos condicionales, en los que todo se resuelve en la etapa ID. Después del salto siempre se busca la instrucción siguiente y en caso de que se produzca el salto se anula la instrucción buscada. El 40 % de los saltos se realizan.
 - El 18% de las veces, las instrucciones $li+1$ tienen una dependencia de LDE con las instrucciones li (el 30 % de éstas corresponden a instrucciones de load).
 - El 6% de las veces las instrucciones $li+2$ tienen dependencias de LDE con la instrucción li , (el 30 % de éstas corresponden a instrucciones de load), y en esos casos nunca hay dependencias entre $li+1$ e $li+2$.

Calcular los ciclos por instrucción para las siguientes características de procesador:

- Procesador sin técnicas para reducir o eliminar paradas. Se puede escribir y leer el registro en el mismo ciclo de reloj.
 - Procesador con forwarding.
 - SPEED UP del segundo caso frente al primero.
- (2,5 ptos)** Se va a ejecutar el código que se muestra a continuación en una máquina con direcciones de 16 bits que dispone de una memoria cache de datos de acceso directo con 4 bloques de 16 bytes cada uno y política de escritura *write-back* con asignación en escritura.
 - Ignorando los accesos a la variable tmp, ¿cuántos fallos de cache se producirán? Justifica tu respuesta, indicando además cuántos reemplazamientos se producen (y en qué iteraciones).
 - Recalcula el número de fallos si la cache fuese asociativa con 2 vías (2 bloques por conjunto) y la política de reemplazamiento fuese LRU.

```
// Cada entero ocupa 4 bytes
int v[16];           // Dirección de comienzo 0x0000
int mask[4];         // Dirección de comienzo 0x0040

1  for(i=0; i<12; i++) {
2      tmp = 0;
3      for(j=0; j<4; j++){
4          tmp = tmp + mask[j]*v[i];
5      }
6      v[i]=tmp;
7  }
```


4. (2,5 ptos) En un procesador segmentado ejecutamos el siguiente fragmento de código (ver tabla adjunta) Se supone que:

- Inicialmente R1=50.
- Un dato se puede escribir en un registro y leer su valor en el mismo ciclo.
- Se dispone de lógica de cortocircuito (forwarding).
- La detección de todo tipo de riesgos (estructurales y LDE) y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos EDE se resuelven mediante inhibición de escritura.
- Dos instrucciones no pueden acceder simultáneamente a la etapa de acceso a memoria ni tampoco a la de escritura en el banco de registros.
- Los saltos se resuelven en la etapa de decodificación y se implementan saltos retardados.

Se dispone de las siguientes unidades funcionales:

UF	Cantidad	Latencia	Segmentación
FP ADDD/ SUBD	1	2	Sí
FP MULD	1	3	Sí
FP DIVD	1	4	No
INT ALU	1	1	No

- Representar el diagrama instrucción-tiempo para la primera iteración e indicar los cortocircuitos realizados así como las paradas producidas y sus causas. A la vista del diagrama obtenido, indicar el número de ciclos por instrucción (CPI) en régimen estacionario.
- Reordenar el código para minimizar las penalizaciones.

5. (2 ptos) Sea un computador con memoria virtual paginada y memoria cache con las características siguientes:

- Memoria física de 8 páginas de 8 KB cada una, con reemplazamiento LRU
- Memoria virtual de 1 MB

Sobre esta jerarquía, se ejecuta un videojuego que necesita realizar una animación de 10 fotogramas. Cada fotograma se muestra con una rutina de dibujo, dicha rutina está almacenada desde la dirección **0A020** y ocupa **2KB**. Cada una de las diez veces que se ejecuta la rutina necesita acceder a 4 imágenes de **4KB**, las imágenes están colocadas en las siguientes direcciones virtuales.

22000, 28000, 2B900 y 0E800

- Indicar el formato de la dirección virtual y de la dirección física
- Considerando que la memoria física está vacía al iniciar por primera vez la rutina de animación. Calcular el total de fallos indicando claramente cómo evolucionan los contenidos en la memoria física (recordar que se emplea la política de reemplazamiento LRU). Además, calcular el tiempo de penalización por acceso a memoria para satisfacer esa cadena de accesos, sabiendo que las penalizaciones por fallo son 700 ciclos.
- Calcular el número de fallos, si manteniendo el tamaño de la memoria física, se reduce el tamaño de página a 4KB.



ESTRUCTURA DE COMPUTADORES
Examen Final Junio - 12 de Junio de 2015

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1. **(4 puntos)** En un procesador segmentado es necesario ejecutar el siguiente código:

```
for ( n = 0; n < 100; n++ ) {  
    for ( k = 0; k < 4; k++ ) {  
        out[n] += filter[k] * input[n+k];  
    }  
}
```

Queriendo conseguir el mayor rendimiento, se decide realizar la siguiente implementación en ensamblador:

```
loop_n : ADDI    R2,R0,#0  
loop_k : LS      F3,0(R3)      ; lee de memoria 4 bytes (load de un float)fa  
        LS      F4,0(R4)      ; lee de memoria 4 bytes (load de un float)  
        MULS    F6,F3,F4      ; mult. en punto flotante (de dos float)  
        LS      F5,0(R5)      ; lee de memoria 4 bytes (load de un float)  
        ADDS    F5,F6,F5      ; suma en punto flotante (de dos float)  
        SS      0(R5),F5      ; escribe en memoria 4 bytes (store de un float)  
        ADDI    R3,R3,#4      ; actualizo puntero a filter  
        ADDI    R4,R4,#4      ; actualizo puntero a input  
        ADDI    R2,R2,#1  
        BLTI    R2,#4, loop_k ; salta a loop_k si R2 < 4  
        ADDI    R5,R5,#4      ; avanzamos el puntero sobre out  
        SUBI    R3,R3,#16     ; R3 vuelve a apuntar a filter[0]  
        SUBI    R4,R4,#12     ; R4 apunta al siguiente input[n]  
        ADDI    R1,R1,#1  
        BLTI    R1,#100,loop_n ; salta a loop_n si R1 < 100
```

Se supone que:

- Inicialmente R1=0, R3 apunta a filter[0], R4 apunta a input[0] y R5 apunta a out[0]
- Un dato se puede escribir en un registro y leer su valor en el mismo ciclo.
- Se dispone de lógica de cortocircuito (*forwarding*).
- La detección de todo tipo de riesgos (estructurales y LDE) y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos EDE se resuelven mediante paradas hasta que la instrucción ya lanzada entre en la etapa de memoria.
- Dos instrucciones no pueden acceder simultáneamente a la etapa de acceso a memoria ni tampoco a la de escritura en el banco de registros.
- Los saltos se resuelven en la etapa de decodificación y NO se implementan saltos retardados (se cancela la siguiente instrucción si el salto es tomado)
- Se dispone de las siguientes unidades funcionales:

UF	Cantidad	Latencia	Segmentación
FP ADDS/ SUBS	1	2	Sí
FP MULS	1	3	Sí
INT ALU	1	1	No

- (2 puntos) Rellena 25 ciclos del diagrama instrucción-tiempo de la hoja adjunta correspondiente **al comienzo del bucle interno con k=3 y n = 0**. Si es necesario para llegar a los 25 ciclos, añade instrucciones a la tabla (filas 16 y 17).
- (0.5 puntos) A partir del apartado anterior, estima el número de ciclos que tardará en ejecutarse el código completo y determina el CPI. **Indica claramente** los cálculos que realices para llegar a los resultados.
- (0.5 puntos) En una versión del procesador, sí se implementan saltos retardados (*delay-slot*). Adapta (si es necesario) el código anterior para que siga funcionando correctamente. ¿Se obtiene algún beneficio para este código?. **Justifica tu respuesta.**
- (1 punto) Asumiendo el procesador original (sin *delay-slot*), trata de mejorar el rendimiento mediante reordenación de instrucciones y/o transformaciones de código a alto nivel. Se valorarán especialmente estas últimas, ya que conducirán a mejor rendimiento. En cualquier caso, escribir el código final en ensamblador y estimar nuevamente los ciclos de ejecución y el CPI.

2.- (3 puntos) En un computador con caches separadas de datos e instrucciones, de 32KB cada una y bloques de 256 bytes, se quiere ejecutar el siguiente código C, que realiza la operación matricial $C = A + B$:

```
#define N 128
int A[N][N];
int B[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = A[i][j] + B[i][j];
```

Asumiendo que la matriz **A** se ubica en la dirección **0x0C000000**, que B y C se almacenan consecutivamente a continuación de A, y que en la traducción a ensamblador del código anterior la variable *i* se ubica en un registro y que las direcciones de comienzo de los *arrays* están también en registro, se pide:

- (0.50 puntos) Calcular el número de fallos de cache en lectura (load) y en escritura (store) si la cache es de Emplazamiento Directo sin asignación en escritura.
- (0.75 puntos) Repetir el cálculo para una cache Asociativa por conjuntos, de 2 vías, con política de reemplazamiento LRU, en los siguientes casos:
 - (0.25 puntos) sin asignación en escritura.
 - (0.5 puntos) con asignación en escritura.

En cada caso indicar si el rendimiento mejoraría o empeoraría y en qué proporción.

- (1 punto) Un programador sugiere la siguiente modificación de código para una cache Asociativa por Conjuntos de 2 vías con asignación en escritura:

```
#define N 128
struct {
    int A;
    int B;
} AB[N][N];
int C[N][N];

for (i=0; i < N; i++)
    for (j=0; j < N; j++)
        C[i][j] = AB[i][j].A +
AB[i][j].B;
```

Asumiendo que el array AB se ubica en la dirección 0x0C000000 y que el array C se ubica a continuación, calcula el número de fallos en lectura y escritura que se producirían, y compáralo con el resultado del apartado b para la misma configuración de cache.

- (0.75 puntos) ¿Puede obtenerse un resultado similar al del apartado c con el código original y sin aumentar el tamaño de la cache? Razona la respuesta.

3.- (3 puntos) Sea un computador con una memoria virtual paginada y memoria cache con las siguientes características:

- Un procesador que genera direcciones de 20 bits
- Una memoria física de 128 KB y páginas de 32 KB. Reemplazamiento LRU
- Una memoria cache de datos de direcciones físicas de 1 KB. La memoria es de acceso directo con 256 B por bloque.

Sobre esta jerarquía se ejecuta una aplicación que selecciona aleatoriamente una canción dentro de una base de datos de canciones. Cada canción ocupa 8 KB y sus direcciones de comienzo están indicadas en la siguiente tabla:

Canción	1	2	3	4	5	6
Dirección	0x50000	0xF4000	0x66000	0x12000	0x40000	0x36000

Se pide:

- (0.25 puntos) Indicar el formato de la dirección virtual y de la dirección física, esta última desde el punto de vista de la memoria virtual y de la memoria cache.
- (0.5 puntos) Indicar en qué página (o páginas) virtuales está ubicada cada canción.
- (1 punto) Si en un momento dado el contenido de la TLB y de la memoria cache de datos es el indicado en las tablas que se muestran a continuación, ¿cuáles han sido las últimas canciones escuchadas? ¿Cuál es el rango de direcciones físicas de cada una de ellas?

Nºpag. virtual	Nº pag. física
0x0A	1
0x0C	2
0x02	0
0x1E	3

TLB

Etiqueta	Bloque
0x70	0
0x27	1
0x27	2
0x27	3

CACHE

- (1,25 puntos) Supongamos que a continuación se genera la siguiente cadena de referencias virtuales: 0xF40F5, 0x66000, 0x40040 y 0x51D04. Indicar si se producen fallos o aciertos de cache y fallos de página, mostrando cómo evolucionan los contenidos de la memoria principal y de la memoria cache.

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	LS F3,0(R3)																									
2	LS F4,0(R4)																									
3	MULS F6,F3,F4																									
4	LS F5,0(R5)																									
5	ADDS F5,F6,F5																									
6	SS 0(R5),F5																									
7	ADDI R3,R3,#4																									
8	ADDI R4,R4,#4																									
9	ADDI R2,R2,#1																									
10	BLTI R2,#4, ...																									
11	ADDI R5,R5,#4																									
12	SUBI R3,R3,#16																									
13	SUBI R4,R4,#12																									
14	ADDI R1,R1,#1																									
15	BLTI R1,100...																									
16																										
17																										

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1																										
2																										
3																										
4																										
5																										
6																										
7																										
8																										
9																										
10																										
11																										
12																										
13																										
14																										
15																										
16																										
17																										

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	LS F3,0(R3)																									
2	LS F4,0(R4)																									
3	MULS F6,F3,F4																									
4	LS F5,0(R5)																									
5	ADDS F5,F6,F5																									
6	SS 0(R5),F5																									
7	ADDI R3,R3,#4																									
8	ADDI R4,R4,#4																									
9	ADDI R2,R2,#1																									
10	BLTI R2,#4, ...																									
11	ADDI R5,R5,#4																									
12	SUBI R3,R3,#16																									
13	SUBI R4,R4,#12																									
14	ADDI R1,R1,#1																									
15	BLTI R1,100...																									
16																										
17																										

		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1																										
2																										
3																										
4																										
5																										
6																										
7																										
8																										
9																										
10																										
11																										
12																										
13																										
14																										
15																										
16																										
17																										



ESTRUCTURA DE COMPUTADORES
Examen Final Junio - 28 de Junio de 2016

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un microprocesador DLX con las siguientes características:

- Segmentado en 5 etapas: IF, ID, EX, MEM, WB.
- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos (i.e. tiene implementado cortocircuito). El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID (para instrucciones de salto).
- Implementa saltos retardados con un "delay-slot" de un ciclo.
- Las dependencias de LDE se detectan en la etapa de decodificación y se resuelven mediante cortocircuito en cuanto están disponibles los operandos.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- Las unidades funcionales con las que cuenta el procesador son las siguientes:

Unidad Funcional	#	Latencia	Segmentada
FP ADD/SUB	1	2	Sí
FP MUL	1	3	Sí
Integer ALU	1	1	No

Se ejecuta el siguiente código sobre dicho procesador. El código trabaja sobre 2 arrays, inicializados a los siguientes valores: A={5'5, 7'2, -2'8, 8'1} y B={-207'8, 9'3, -3'5, 13'4}, y almacena el resultado en un tercer vector C. **Este código se ha obtenido usando un compilador sin optimización, de forma que implementa saltos retardados introduciendo NOP detrás de las instrucciones de salto.**

```
LOOP:      LD      F1, 0 (R3)
           LD      F2, 0 (R4)
           MULF   F3, F1, F2
           ADDI   R4, R4, #8
           ANDI   R6, R1, #1
           BEQ    R6, R0, NO_SUMAR
           NOP
           ADDF   F3, F3, F4

NO_SUMAR:  ADDI   R1, R1, #1
           SUBF   F3, F3, #4
           SD     F3, 0 (R5)
           ADDI   R3, R3, #8
           ADDI   R5, R5, #8
           BNE    R1, R2, LOOP
           NOP
```

Se supone que los siguientes registros están inicializados al valor indicado a continuación:

- R0=0
 - R1=0
 - R2=4
 - R3= Dirección inicial del vector A
 - R4= Dirección inicial del vector B
 - R5= Dirección inicial del vector C
 - F4=1
- a) (2.25 puntos) Representar el diagrama instrucción-tiempo para las dos primeras iteraciones e indicar los cortocircuitos realizados. Indicar claramente las paradas y sus causas.
- b) (0.75 puntos) Calcular el CPI.
- c) (0.5 puntos) ¿Cómo habría quedado el código si se hubiera usado un compilador que implementa de manera más óptima los saltos retardados? ¿Cómo afecta esto al CPI? Justificar la respuesta.
- d) (0.5 puntos) Reordenar el código para que el CPI sea mínimo, incluyendo las modificaciones hechas en el apartado anterior si las hubiera. No es necesario realizar de nuevo el cronograma ni calcular el CPI.

2.- (3 puntos) Sea un computador con una memoria principal de 256KB, direccionable en bytes, al que se dota de una memoria de cache de 128 bytes, con bloques de 16 bytes. Se quiere ejecutar el siguiente código C:

```
int A[16][16];
int B[32];
int C[16][16];

for (i=0; i< 16; i++)
    C[0][i] = A[0][i] + B[4];
```

Asumiendo que **la matriz A se ubica en la dirección 0x10000**, que B y C se almacenan consecutivamente a continuación de A, y que en la traducción a ensamblador del código anterior la variable i y las direcciones de comienzo de A, B y C se ubican en registro, se pide:

- (1 punto) Calcular el número de fallos de cache en lectura (load) y en escritura (store) si la cache es de emplazamiento directo, con asignación en escritura (write-allocate). **Suponer que en cada iteración el primer elemento que se lee de memoria es B[4].**
- (1 punto) Repetir el cálculo para una cache asociativa por conjuntos, de 2 vías, con política de reemplazamiento FIFO y con asignación en escritura.
- (0.5 puntos) ¿Puede reducirse el número de fallos con el código original y sin aumentar el tamaño de la cache? Razona la respuesta.
- (0.5 puntos) Considerar la MC del apartado a), si el tiempo de acceso a una palabra en cache es 2 ns y el tiempo de penalización por fallo es de 150ns, tanto para fallo de lectura como para fallo de escritura, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?

3.- (3 puntos) Sea un computador con una memoria virtual paginada y memoria cache con las siguientes características:

- Sistema de memoria virtual de 4GBytes.
- Una memoria física de 64MB y páginas de 8MB que tiene implementada la política de reemplazamiento FIFO
- Una cache de datos de 256KB, físicamente accedida, con emplazamiento directo. El tamaño del bloque es de 64KB

En el disco duro del procesador están almacenados 20 álbumes de fotos (Alb1, ... , Alb20). Se ejecuta una aplicación que va seleccionando **aleatoriamente** uno de estos álbumes y muestra por pantalla, de manera aleatoria, las fotos que contiene. Cada álbum ocupa 2MB y cada foto 64KB. **El Alb1 empieza en la dirección virtual 0x02800000 y los demás están en direcciones virtuales consecutivas.**

Se pide:

- (0.5 puntos) Indicar el formato de la dirección virtual y de la dirección física, esta última desde el punto de vista de la memoria principal y de la memoria cache.

Sabiendo que en un momento de la ejecución el contenido de la tabla de páginas, de la TLB y de la caché de datos es el que se muestra en las siguientes tablas:

- (0.75 puntos) Indica que álbumes están en ese momento en la memoria principal
- (0.75 puntos) En qué direcciones físicas empiezan los álbumes de la última página que se colocó en memoria principal

Nº fila	V	FIFO	Nº pag. física
.	0	.	.
.	.	.	.
.	0	.	.
5	1	1	6
6	1	0	0
.	0	.	.
.	0	.	.
9	1	2	1
.	0	.	.
.	.	.	.
.	0	.	.

FIFO	V	Nº pag. virtual	Nº pag. física
0	1	5	6
1	1	9	1

Etiqueta	Bloque
0xD0	0
0xD5	1
0xD0	2
0x27	3

Toda la información está expresada en decimal, salvo la etiqueta de la caché.

V es el bit de validez también llamado Hit

El bit de FIFO indica la entrada del elemento de memoria en orden creciente

- (1 punto) Si en un momento dado se accede a la foto que está en la dirección virtual 0x03400000 indica en qué orden se accede a las distintas componentes del sistema de memoria, si se produce fallo o acierto en cada uno de ellos y cómo quedaría su contenido.



ESTRUCTURA DE COMPUTADORES
Examen Final Junio - 22 de Junio de 2017

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un DLX segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos (tiene implementado cortocircuito).
- El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID (para instrucciones de salto).
- La detección de riesgos LDE y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera en la última etapa de cada unidad funcional.
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- Los saltos se resuelven en la etapa de decodificación y NO se implementan saltos retardados (se cancela la siguiente instrucción si el salto es tomado).
- Las unidades funcionales del procesador son:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	3	Sí
FP MUL	1	5	Sí
Int ALU	1	1	No

Sabiendo que el siguiente fragmento de código se ejecuta sobre dicho procesador

```
Bucle:   BNE R1,R0, Nosuma
         LD F1,0(R5)
         ADDI R1,R1,1
Nosuma: LD F5,0(R2)
         MULF F4,F5,F1
         LD F6,8(R2)
         MULF F4,F6,F4
         ADDD F1, F4, F1
         SUB R6,R1,R3
         ADDI R2,R2,16
         ADDI R3,R3,1
         BNE R3,R4,Bucle
Finbucle: SD F1,0(R5)
```

Suponiendo que los siguientes registros están inicializados al valor indicado a continuación:

- R0=0
 - R1=0
 - R2= Dirección inicial del vector
 - R3= 0
 - R4= 10
 - R5=Dirección del resultado
- a) (2 puntos) Representar el diagrama instrucción-tiempo para la primera iteración e indicar los cortocircuitos realizados. Indicar claramente las paradas y sus causas.
- b) (0.75 puntos) Calcular el CPI de la **ejecución completa del código**
- c) (0,75 puntos) Reordenar el código para reducir las paradas.
- d) (0.5 punto) Si tuviese saltos retardados ¿cómo habría que cambiar el **código no reordenado** para que se ejecutase correctamente?

2.- (3 puntos) Sea un computador con un sistema de memoria con las siguientes características:

- Una memoria principal de 64KB
- Una cache de datos de 64Bytes, con emplazamiento directo, con asignación en escritura (write-allocate) y política de actualización post-escritura
- El tamaño del bloque es de 16Bytes

Se quiere ejecutar el siguiente código C:

```
int A[5][8];
int B[2][8];
int C[5];

for (i=0; i<5; i++)
{
    suma =0;
    for (j=0; j<8; j++)
    {
        if i<2 then
            suma = suma + A[i][j] + B[i][j];
        else
            suma = suma + A[i][j];
    }
    C[i]= suma ;
}
```

Asumiendo que la matriz **A** se ubica en la dirección **0x0010**, que B y C se almacenan consecutivamente a continuación de A, y que en la traducción a ensamblador del código anterior las variables i, j, suma y las direcciones de comienzo de A, B y C se ubican en registros, se pide:

- (0.5 puntos) Indicar el rango de direcciones en las que se encuentran las variables A, B y C. ¿Cuántos bloques de memoria principal y cuáles ocupan cada una de las variables A, B y C?
- (0.75 puntos) ¿Qué contenido (etiqueta, elementos de A, B ó C que están almacenados en el bloque cache y el número de bloque de MP al que corresponde) tiene la caché cuando acaba la vuelta i= 0 ?
- (1 punto) Calcular el número de fallos de cache en lectura (load) y en escritura (store) que se obtienen en la ejecución de ese programa.
- (0.75 puntos) Para mejorar la tasa de fallos se decide implementar prebúsqueda bajo fallo (en caso de fallo se trae el bloque que lo provoca y el siguiente a la cache). Esta mejora ¿reduce la tasa de fallos? Si es así calcular el número de fallos de cache en lectura (load) y en escritura (store).

3.- (3 puntos) Sea un computador que genera direcciones de 32 bits con una memoria virtual paginada que tiene las siguientes características:

- Tamaño de página de 4KB
- Una memoria física de 1GB
- Una cache virtualmente accedida físicamente marcada de 2KB con política de emplazamiento directo. El tamaño del bloque de 512B
- Una TLB asociativa de dos entradas con reemplazamiento LRU

En este sistema el proceso activo está ejecutando un código de procesamiento de vídeo que utiliza **20 fotogramas** (fg0-fg19) de **1KB ubicados consecutivamente** en el espacio virtual a partir de la dirección **0x0C000800**. En ese momento el contenido de la tabla de páginas del proceso, la TLB y el directorio de la cache son los siguientes:

Nº fila	V	Nº pag. Física
0x00000	0	
...		
0x0C000	1	0x00004
0x0C001	1	0x00002
...		
0x0C005	1	0x00600
0x0C006	1	0x00601
...		
0x0C100	1	0x10000
0x0C101	1	0x10001
...		
0x7FFFF	1	0x04000
...		

LRU	V	Nº pag. virtual	Nº pag. física
0	1	0x0C001	0x00002
1	1	0x0C000	0x00004

V	Etiqueta	Bloque
1	0x00004	0
1	0x00004	1
1	0x00009	2
1	0x00009	3

El bit de LRU con valor 0 se corresponde con el más recientemente accedido

Se pide:

- a) (0.5 puntos) Indicar los formatos de las direcciones virtual y física, esta última desde el punto de vista de la gestión de la memoria virtual y de la gestión de la memoria cache.

NOTA: en las caches virtualmente accedidas físicamente marcadas el número de bits de los campos bloque de cache y byte del bloque es como máximo el número de bits del campo de offset (no tienen por qué coincidir)

- b) (0.75 puntos) Indicar qué fotogramas están actualmente en memoria principal, y cuáles son sus direcciones físicas de comienzo.
- c) (0.75 puntos) Indicar cuáles son los fotogramas que se han accedido más recientemente. ¿De estos cuál es el último en acceder?
- d) (1 punto) Si en este momento el procesador direcciona la primera palabra del fotograma 18, indicar en qué orden se accede a cada uno de los elementos que componen el sistema de memoria, indicando la dirección virtual o física con la que se accede y el bloque y la etiqueta en el caso de la cache. Indicar si se produce fallo o acierto en cada uno de ellos y cómo quedarían tras el acceso.

	ESTRUCTURA DE COMPUTADORES Examen Final Septiembre - 9 de Septiembre de 2014
	Nombre _____ DNI _____ Apellidos _____ Grupo _____

PUNTUACIÓN: el problema 1 vale 2 puntos; los problemas 2 y 4 valen 2,5 cada uno; el problema 3 vale 3 puntos.

- Sea un sistema formado por un bus síncrono con líneas dedicadas. El bus tiene un ancho de palabra de 16 bits y un reloj de 100MHz.
 - Si al bus se conecta un periférico con un ancho de banda de 1,6MB/s, ¿cuánto se tarda en transferir 1KB?
 - Si se quiere conectar al bus un dispositivo con gran ancho de banda será necesario que el dispositivo disponga de un buffer para ir almacenando datos hasta que el bus esté disponible para enviar un nuevo dato. ¿Cuál es el ancho de banda máximo que puede tener un periférico conectado a este bus para que no sea necesario un buffer?
 - Si se conecta un periférico (con buffer) con un ancho de banda de 240 MB/s, ¿cuánto tiempo tardará el sistema en enviar 64 palabras de 16 bits (no se espera a tener las 64 palabras para empezar el envío)? ¿De qué tamaño debe ser el buffer, si las transferencias son siempre de 64 palabras, para no perder ninguna?

- En un procesador segmentado ejecutamos el siguiente fragmento de código:

```

Loop :   LD   F1,0(R3)
         DIVD F0,F1,F2
         SUBD F3,F0,F1
         DIVD F4,F4,F2
         DIVD F2,F0,F1
         ADDD F2,F6,F0
         LD   F4,0(R3)
         SUBI R3,R3,#1
         SD   0(R1),F4
         SUBI R1,R1,#1
         BNEZ R1,Loop
         MULD F6,F0,F0
         ADDD F3,F2,F2

```

Se supone que:

- Inicialmente R1=100.
- Un dato se puede escribir en un registro y leer su valor en el mismo ciclo.
- Se dispone de lógica de cortocircuito (*forwarding*).
- La detección de todo tipo de riesgos (estructurales y LDE) y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos EDE se resuelven mediante paradas hasta que la instrucción ya lanzada entre en la etapa de memoria.
- Dos instrucciones no pueden acceder simultáneamente a la etapa de acceso a memoria ni tampoco a la de escritura en el banco de registros.
- Los saltos se resuelven en la etapa de decodificación y se implementan saltos retardados.
- Se dispone de las siguientes unidades funcionales:

UF	Cantidad	Latencia	Segmentación
FP ADDD/ SUBD	1	2	Sí
FP MULD	1	3	Sí
FP DIVD	1	4	No
INT ALU	1	1	No

Se pide representar el diagrama instrucción-tiempo para la primera iteración e indicar los cortocircuitos realizados así como las paradas producidas y sus causas. A la vista del diagrama obtenido, indicar el número de ciclos por instrucción (CPI) en régimen estacionario.

3. Se va a ejecutar el código que se muestra a continuación en una máquina con direcciones de 16 bits que dispone de una memoria cache de datos de acceso directo con 8 bloques de 16 bytes cada uno y política de escritura *write-through* (las escrituras no se realizan en la cache) y sin asignación en escritura (una escritura nunca supone llevar un nuevo bloque a cache). Teniendo en cuenta sólo la cache de datos (es decir, ignorando la lectura de instrucciones):

- Indica la dirección de memoria (expresada en hexadecimal) del elemento $M[0][1]$. ¿A qué (marco de) bloque de la memoria cache se llevará dicho elemento cuando se emplace en la cache?
- Ignorando los accesos a las variables tmp , i , j , ¿cuántos fallos de cache se producirán? Justifica tu respuesta, indicando además cuántos reemplazamientos se producen (y en qué iteraciones).
- Propón una modificación de código que minimice los fallos por conflicto para la misma cache usada en el apartado anterior.

```
// Cada entero ocupa 4 bytes
int Vin[16];           // Dirección de comienzo 0x0000
int Vout[16];          // Dirección de comienzo 0x0040
int M[16][16];         // Dirección de comienzo 0x0080

1  for(i=0; i<4; i++) {
2      tmp = 0;
3      for(j=0; j<16; j++){
4          tmp = tmp+Vin[j]*M[j][i];
5      }
6      Vout[i]=tmp;
7  }
```

4. Sea un computador con memoria virtual paginada y sin memoria cache con las características siguientes: memoria física de 4 páginas de 8 KB cada una, con reemplazamiento LRU y memoria virtual de 1 MB. Sobre esta jerarquía, se ejecuta una aplicación de contabilidad que contiene la facturación de los últimos 4 meses. El volumen de datos de cada mes y su posición es diferente, y corresponden a:

	DATOS				Programa de contabilidad
	Enero	Febrero	Marzo	Abril	
Tamaño	8KB	4KB	1KB	2KB	16KB
Dirección virtual de comienzo (hex)	13000	16000	A0000	AB000	20000

- Indica el formato de la dirección virtual y de la dirección física.
- Indica las páginas virtuales que les corresponden a cada uno.
- La empresa tiene 10 agentes comerciales. Se quiere que el programa de contabilidad calcule el total de ventas de cada agente durante los 4 meses del año según la rutina siguiente:

```
void programa_de_contabilidad()
{
    int comercial, mes;
    int ventas[10]= {0,0,0,0,0, 0,0,0,0,0};
    for (comercial =0 ; comercial < 10; comercial++){
        for(mes =1; mes <=4; mes++){
            ventas[comercial] += calculo_ventas(comercial,mes);
        }
    }
}
```

Considera que: la memoria física está vacía al inicio del programa; que cuando la rutina accede a los datos de un mes carga en memoria todas las páginas que ocupen los datos de ese mes; que todas las páginas del programa están siempre en uso (se consideran siempre más recientemente accedidas que las páginas que contienen datos).

Calcula el total de fallos para satisfacer esa cadena de accesos. Indica claramente cómo evolucionan los contenidos en la memoria física.



ESTRUCTURA DE COMPUTADORES
Examen Final Septiembre – 8 de Septiembre de 2015

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1. (3 puntos) Sea un sistema de memoria con las siguientes características:
- Memoria virtual paginada de 16MB, política de emplazamiento LRU
 - Tamaño de página 4KB
 - Memoria principal con 32KB
 - Memoria cache de direcciones físicas 8Kbytes, con bloques de 64 bytes, asociativa por conjuntos con 2 bloques/cjtos, política LRU

Tabla de paginas asociativa

Nº pagina	Nº de marco	Último acceso
100	0	0
101	1	1
200	2	2
201	3	3
300	4	4
301	5	5
400	6	6
401	7	7

- a) (0,5 puntos) Indicar el formato de la dirección virtual y de la dirección física, esta última desde el punto de vista de la memoria virtual y de la memoria cache.

Se desea acceder ejecutar el siguiente código:

```
int A[16384];

for (i=0; i<16384; i++)
    red+= A[i];
```

Suponiendo que A está almacenado consecutivamente en la dirección virtual 0x100000 y que la variable red está almacenada en un registro y únicamente se lleva a memoria al finalizar la ejecución del bucle

- b) (1,25 puntos) ¿Cuántas páginas de memoria virtual se visitan? ¿Cuáles son?
c) (1,25 puntos) Razonar el número de aciertos y fallos de página que se producen.

2. (3 puntos) Considerar un computador con direcciones de 32 bits, al que se dota de una memoria cache de 128 bytes, con bloques de 16 bytes y emplazamiento directo, sin asignación en escritura (No write-allocate).

Suponer que la variable a está en el banco de registros y no en cache.

Se quiere ejecutar el siguiente código:

```
int nota[128]; // nota[0] está en la dirección 0x00000000
int media[128]; // media [0] está a continuación de nota[127]

for (i=0;i<128;i++) {
    if (i>7 && i<64) {
        nota[i] = media[i]/2;
    }
    else {
        a = nota[i]*media[i]
    }
}
```

- a) (1 punto) Indicar cuántos fallos de cache se producen.
b) (1 punto) Proponer dos optimizaciones de código (independientes) que mejoren el resultado, indicando cuántos fallos se producen en esos casos.
c) (1 punto) Si el tiempo de acceso a una palabra en cache es 1 ns, el tiempo de lectura o escritura de una palabra en memoria principal es 50 ns, el tiempo de transferencia de un bloque de MP a cache es 200 ns, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa?

3. (4 puntos) Un procesador segmentado genera una imagen en escala de grises a partir de otras 2 imágenes A y B también en escala de grises, mediante el código siguiente:

Para facilitar el cálculo, los píxeles de las imágenes A y B están almacenados en posiciones alternas en memoria, de forma que los píxeles de la imagen A están en posiciones pares, y los de la B en posiciones impares.

```
for ( fila = 0; fila < 100; fila++ ) {
    for ( col = 0; col < 100; col++ ) { //numcol=100
        imagengris[fila*numcol+col] = 0.43*imagenesAB[(fila*numcol+col)*2]; //componente img A
        imagengris[fila*numcol+col] += 0.57*imagenesAB[(fila*numcol+col)*2+1]; //componente img B
    }
}
```

Nótese, que en el código en C indica entre corchetes el elemento en coma flotante a acceder, y no indica el valor de la dirección en memoria. El float [n+1] está 4 bytes después del float [n]

Se ha realizado la siguiente implementación en ensamblador:

```

SUBI    R1,R1,#0    ; R1 contador de fila
Loop_fil: SUBI    R2,R2,#0    ; R2 contador de columna
Loop_col: MUL    R3,R1,R4
        ADD    R3,R3,R2    ; n= fila*numcol+col
        MULI   R6,R3,#2    ; elemento floatn= (fila*numcol+col)*2
        MULI   R6,R6,#4    ; desplazamiento en bytes del elemento float n
        ADD    R6,R6,R5    ; dirección del elemento imagenesAB[(fila*numcol+col)*2]
        LS     F1,0(R6) ; lee de memoria 4 bytes (load de un float)
        MULD   F1,F1,F4
        LS     F2,4(R6) ; lee de memoria 4 bytes (load del siguiente float)
        MULD   F2,F2,F5
        ADDDD  F1,F1,F2
        ADD    R8,R3,R7    ; dirección del elemento imagengris[fila*numcol+col]
        SS     O(R8),F1; escribe en memoria 4 bytes (store de un float)
        ADDI   R2,R2,#1    ; avanzamos a la siguiente columna
        BLTI   R2,#100,loop_col ; salta a loop_n si R2 < 100
        ADDI   R1,R1,#1    ; avanzamos a la siguiente fila
        BLTI   R1,#100,loop_fil ; salta a loop_n si R1 < 100
```

Se supone que:

- Inicialmente F4= 0.43 y F5=0.57
- Inicialmente R4=numcol, R5 apunta a la dirección de inicio imagenesAB[0], y R7 apunta a la dirección de imagengris[0]
- Un dato se puede escribir en un registro y leer su valor en el mismo ciclo.
- Se dispone de lógica de cortocircuito (*forwarding*).
- La detección de todo tipo de riesgos (estructurales y LDE) y generación de paradas se realiza en la etapa de decodificación.
- Los riesgos EDE se resuelven mediante paradas hasta que la instrucción ya lanzada entre en la etapa de memoria.
- Dos instrucciones no pueden acceder simultáneamente a la etapa de acceso a memoria, ni tampoco a la de escritura en el banco de registros.
- Los saltos se resuelven en la etapa de decodificación y NO se implementan saltos retardados (se cancela la siguiente instrucción si el salto es tomado)
- Se dispone de las siguientes unidades funcionales:

UF	Cantidad	Latencia	Segmentación
FP ADDD/ SUBD	1	2	Sí
FP MULD	1	3	Sí
INT ALU	1	1	No

- (2 puntos) Rellena las 16 filas del diagrama instrucción-tiempo de la hoja adjunta correspondiente **al comienzo del bucle interno con FILA=10 y COL = 0.**
- (0.5 puntos) A partir del apartado anterior, estima el CPI que se logrará en la ejecución del código completo.
- (1 pt) Minimizar el CPI mediante reordenación de instrucciones y/o transformaciones de código a alto nivel. Se valorarán especialmente estas últimas. **OBLIGATORIO**, escribir el código final en ensamblador.
- (0.5 pts) Estima el CPI en la ejecución del código original en un procesador mejorado en el que todas las operaciones de coma flotante necesitan de solo un ciclo en la ALU (latencia 1). Indica el speedup respecto del apartado b.

			1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
1	Loop_col:	MUL R3,R1,R4																											
2		ADD R3,R3,R2																											
3		MULI R6,R3,#2																											
4		MULI R6,R6,#4																											
5		ADD R6,R6,R5																											
6		LS F1,0(R6)																											
7		MULD F1,F1,F4																											
8		LS F2,4(R6)																											
9		MULD F2,F2,F5																											
10		ADDD F1,F1,F2																											
11		ADD R8,R3,R7																											
12		SS O(R8),F1																											
13		ADDI R2,R2,#1																											
14		BLTI R2,#100,loop_col																											
15																													
16																													

			1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
1	Loop_col:	MUL R3,R1,R4																											
2		ADD R3,R3,R2																											
3		MULI R6,R3,#2																											
4		MULI R6,R6,#4																											
5		ADD R6,R6,R5																											
6		LS F1,0(R6)																											
7		MULD F1,F1,F4																											
8		LS F2,4(R6)																											
9		MULD F2,F2,F5																											
10		ADDD F1,F1,F2																											
11		ADD R8,R3,R7																											
12		SS O(R8),F1																											
13		ADDI R2,R2,#1																											
14		BLTI R2,#100,loop_col																											
15																													
16																													

			1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
1	Loop_col:	MUL R3,R1,R4																											
2		ADD R3,R3,R2																											
3		MULI R6,R3,#2																											
4		MULI R6,R6,#4																											
5		ADD R6,R6,R5																											
6		LS F1,0(R6)																											
7		MULD F1,F1,F4																											
8		LS F2,4(R6)																											
9		MULD F2,F2,F5																											
10		ADDD F1,F1,F2																											
11		ADD R8,R3,R7																											
12		SS O(R8),F1																											
13		ADDI R2,R2,#1																											
14		BLTI R2,#100,loop_col																											
15																													
16																													

			1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
1	Loop_col:	MUL R3,R1,R4																											
2		ADD R3,R3,R2																											
3		MULI R6,R3,#2																											
4		MULI R6,R6,#4																											
5		ADD R6,R6,R5																											
6		LS F1,0(R6)																											
7		MULD F1,F1,F4																											
8		LS F2,4(R6)																											
9		MULD F2,F2,F5																											
10		ADDD F1,F1,F2																											
11		ADD R8,R3,R7																											
12		SS O(R8),F1																											
13		ADDI R2,R2,#1																											
14		BLTI R2,#100,loop_col																											
15																													
16																													



ESTRUCTURA DE COMPUTADORES
Examen Final - 8 de Septiembre de 2016

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un microprocesador DLX segmentado en 5 etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos (tiene implementado cortocircuito).
- El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID (para los saltos).
- Implementa saltos retardados con un “delay-slot” de un ciclo.
- Las dependencias de LDE se detectan en la etapa de decodificación y se resuelven mediante cortocircuito en cuanto los operandos están disponibles.
- Los riesgos estructurales referidos a memoria se detectan y se resuelven mediante espera de la segunda operación en la etapa de decodificación.
- Los riesgos de EDE se resuelven mediante inhibición de escritura.
- Las unidades funcionales con las que cuenta el procesador son las siguientes:

Unidad Funcional	#	Latencia	Segmentada
FP ADD	1	3	Sí
FP MUL	1	5	Sí
FP DIV	1	6	No
Integer ALU	1	1	No

Se ejecuta el siguiente código, suponiendo R0=0.

```
.data    0x2000
n:       .word 8
m:       .word 0
datosX:  .double 2.1, 5.4, 3.9, 4.3, 5.1, 7.6, 2.7, 0.8
datosY:  .double 6.3, 9.7, 3.9, 2.1, 7.4, 6.8, 2.5, 8.4
datosZ:  .space 64
        .text 0x100
lw R2, m(R0)           ; R2 ← MEM[R0 + dirección de m]
beqz R2, L0
nop
addi R2, R2, 4
sw m(R0), R2           ; MEM[R0 + dirección de m] ← R2
L0:     addi R1, R0, 0
        lw R8, n(R0)           ; R8 ← MEM[R0 + dirección de n]
        ld F0, datosX(R1)       ; F0 ← MEM[R1 + dirección de comienzo de datosX]
        ld F2, datosY(R1)       ; F2 ← MEM[R1 + dirección de comienzo de datosY]
        add F4, F0, F2
        multd F4, F4, F4
        sd datosZ(R1), F4       ; MEM[R1 + dirección de comienzo de datos] ← F4
        subi R8, R8, 1
        bnez R8, eti
        addi R1, R1, 8
eti:
```

- a) (1.5 puntos) Completar el diagrama instrucción-tiempo para los 27 primeros ciclos indicando los cortocircuitos realizados, las paradas y sus causas.
- b) (0.75 puntos) Calcular el CPI.
- c) (0.5 puntos) ¿Sería posible sustituir la operación NOP por alguna otra operación del programa dado? En caso afirmativo, ¿cuál sería la primera operación del código que podría reemplazar a la NOP?
- d) (1.25 puntos) Añadir el siguiente fragmento de código al cuerpo del bucle (a partir de la etiqueta *eti*), reordenando las instrucciones para minimizar el número de paradas. Reescribir el diagrama instrucción-tiempo para el bucle resultante, suponiendo que:

- El vector datosZ se encuentra inicializado en el programa principal.
- En cada iteración del bucle la carga del registro F6 debe hacerse antes de incrementar el valor de R1.
- En cada iteración del bucle la instrucción de escritura (sd) en datosZ debe ser posterior a su lectura (ld).
- El código resultante debe respetar las dependencias de datos existentes tanto en el bucle original como en el fragmento de código dado.

```
ld F6, datosZ(R1)
add F8, F8, F6
multd F10, F10, F6
multd F12, F12, F6
divd F14, F14, F6
divd F16, F16, F6
subi R2, R2, 2
sw m(R0), R2
```

2.- (3 puntos) Sea un computador con una memoria principal de 1MB, direccionable en bytes, al que se dota de una memoria de cache de 256 bytes, con bloques de 32 bytes y reemplazamiento FIFO. Se quiere ejecutar el siguiente código C:

```
int A[64];
int B[64];
int C[32];

for (i=0; i < 32; i++)
    C[i] = A[2*i] + B[2*i];
```

Asumiendo que la matriz **A se ubica en la dirección 0x800**, y que B y C se almacenan consecutivamente a continuación de A, que en la traducción a ensamblador del código anterior la variable *i* y las direcciones de comienzo de A, B y C se ubican en registro, y que en cada iteración se lee primero B[2*i], se pide:

- (1 punto) Calcular el número de fallos de cache en lectura (load) y en escritura (store) si la cache es de emplazamiento directo, con asignación en escritura (write-allocate).
- (1 punto) Calcular el número de fallos de cache en lectura (load) y en escritura (store) si la cache es asociativa por conjuntos de dos vías, con asignación en escritura (write-allocate).
- (0.5 puntos) Describir razonadamente cómo pueden eliminarse todos los fallos por conflicto (quedando sólo fallos de inicio) mediante alguna modificación hardware en la cache que no suponga aumentar su tamaño. Indicar el número de fallos que se producirían en este caso.
- (0.5 puntos) Si el tiempo de acceso a una palabra en cache es 2 ns y el tiempo de penalización por fallo es de 150ns, y suponiendo que las escrituras no tienen coste porque se solapan con las lecturas, ¿cuál es el tiempo necesario para acceder a todas las referencias del programa en los 3 casos anteriores? Indicar la ganancia de los apartados b y c respecto del a.

3.- (3 puntos) Sea un computador con una memoria virtual paginada y memoria cache con las siguientes características:

- Un procesador que genera direcciones de 16 bits
- Una memoria física de 16 KB y páginas de 4 KB. Reemplazamiento LRU
- Un TLB de 4 entradas totalmente asociativo.
- Una memoria cache de datos de direcciones físicas de 1 KB. La memoria es asociativa por conjuntos de dos vías con 256 B por bloque. Reemplazamiento LRU.

Sobre esta jerarquía se ejecuta una aplicación que busca información sobre una ciudad consultando una base de datos de mapas, que contiene los mapas de cuatro ciudades. El código de la aplicación ocupa 256 B y su dirección inicial es 0xC000. El tamaño y dirección inicial de los mapas están indicados en la siguiente tabla:

Mapa	Madrid	Barcelona	Valencia	Bilbao
Dirección	0x2000	0xE000	0x4000	0xA000
Tamaño	6 KB	6 KB	3 KB	3 KB

Se pide:

- (0.5 puntos) Indicar el formato de la dirección virtual y de la dirección física, esta última desde el punto de vista de la memoria principal y de la memoria cache.
- (0.5 puntos) Indicar en qué página (o páginas) virtuales están ubicadas la aplicación y los mapas.
- (1 punto) Cuando la aplicación empieza a ejecutarse se consulta el mapa de Madrid, después el de Valencia y finalmente el de Bilbao. Indicar el contenido del TLB durante esa secuencia de accesos considerando que el TLB y la memoria estaban inicialmente vacías. Considerar que cuando la aplicación usa un mapa lo trae entero a memoria principal y que el código de aplicación está siempre en uso (considerándose siempre su página más recientemente accedida que las páginas que contienen datos).
- (1 punto) Si en un momento dado el contenido de la TLB y de la memoria cache de datos es el indicado en las tablas que se muestran a continuación, y se produce la siguiente cadena de referencias virtuales para el acceso a los datos (no se incluyen las referencias a instrucciones), indicar si se producen fallos o aciertos de cache de datos y fallos de página, mostrando cómo evolucionan los contenidos de la memoria principal, del TLB y de la cache: 0xA40C, 0x41E0, 0xE110.

Nº pag. virtual	Nº pag. física	LRU
0xC	0	0
0xA	2	3
0xF	1	2
0x4	3	1

TLB (el más reciente es el de LRU menor)

Conjunto	Etiqueta	LRU
0	0x17	1
	0x12	0
1	0x17	0
	0x18	1

CACHE (el más reciente es el de LRU menor)



ESTRUCTURA DE COMPUTADORES
Examen Final - Septiembre de 2017

Nombre _____ DNI _____
Apellidos _____ Grupo _____

1.- (4 puntos) Sea un DLX segmentado de cinco etapas con las siguientes características:

- Un dato se puede leer y escribir en el banco de registros en el mismo ciclo de reloj.
- Existe anticipación de operandos (tiene implementado cortocircuito).
- El cortocircuito se realiza siempre desde los registros del pipeline, y existe cortocircuito a la etapa ID (para instrucciones de salto).
- La detección todo tipo de riesgos (LDE y estructurales) y generación de paradas se realiza en la etapa de decodificación
- Los riesgos de EDE entre dos instrucciones A y B tal que A precede a B se resuelven mediante inhibición de escritura de la instrucción A.
- Los saltos se resuelven en la etapa de decodificación y **se implementan saltos retardados**
- Las unidades funcionales del procesador son:

UF	Cantidad	Latencia	Segmentación
FP ADD	1	3	SÍ
FP MUL	1	4	NO
Int ALU	1	1	NO

Sabiendo que el siguiente fragmento de código de alto nivel se ejecuta sobre dicho procesador

```
for ( i = 0; i < 10; i++ ) {  
    tmp=C[i];  
    if ( j!=0 ) j--;  
    C[i] = A[2*i] * tmp + A[2*i+1]*tmp;  
}
```

Queriendo conseguir el mayor rendimiento, se decide realizar la siguiente implementación en ensamblador:

```
Bucle: LD F1,0(R5)  
      BEQ R1,R0, Noresta  
      LD F5,0(R2)  
      SUBI R1,R1,1  
Noresta:LD F6,8(R2)  
      MULF F7,F6,F1  
      MULF F4,F5,F1  
      ADDF F1, F7, F4  
      SDF1,0(R5)  
      ADDI R5,R5,4  
      ADDI R3,R3,1  
      BNE R3,R4,Bucle  
      ADDI R2,R2,16
```

Se supone que los siguientes registros están inicializados al valor indicado a continuación:

- | | |
|--------------------------------------|-------------------------------------|
| ▪ R0=0 | ▪ R5=Dirección inicial del vector C |
| ▪ R1=2 (índice j) | ▪ F1=tmp |
| ▪ R2= Dirección inicial del vector A | ▪ F5=A[2*i] |
| ▪ R3= 0 | ▪ F6=A[2*i+1] |
| ▪ R4= 10 | |

- a) (2 puntos) Representar el diagrama instrucción-tiempo para la primera iteración e indicar los cortocircuitos realizados. Indicar claramente las paradas y sus causas.
- b) (0.75 puntos) Calcular el CPI de la ejecución completa del código.
- c) (0.25 puntos) Dado que este procesador implementa salto retardado, ¿sería posible mover la instrucción SUBI R1,R1,1 debajo de la instrucción de salto BEQ R1,R0, No resta? Justifica la respuesta.
- d) (1 punto) Mejorar el rendimiento mediante reordenación de instrucciones y/o modificaciones del código de alto nivel.

Se valorarán especialmente estas últimas, ya que conducirán a mejor rendimiento. En cualquier caso, escribir el código final en ensamblador y estimar nuevamente los ciclos de ejecución y el CPI.

2.- (3 puntos) El siguiente código calcula los elementos de un vector destino **C[256]** a partir de los elementos de dos vectores origen **A[256]** y **B[256]**.

```
MOV    R5, #256
MOV    R4, #0
MOV    R3, #0
Loop:
LDR     R0, [R6]
AND     R0, R0, #1
CMP     R0, R3
BEQ     NoAct
LDR     R0, [R7]
SDR     R0, [R8]
NoAct:  ADD    R6, R6, #4
        ADD    R7, R7, #4
        ADD    R8, R8, #4
        ADD    R4, R4, #1
        CMP    R4, R5
        BNE    Loop
```

Se nos proporcionan las siguientes especificaciones.

- El tamaño de las instrucciones es de 4 bytes y los elementos de A, B y C son números enteros.
- El código está ubicado a partir de la dirección de memoria 0x80000000. El vector A está ubicado a partir de la dirección de memoria 0x80001000, el vector B se encuentra almacenado a continuación de A, y el vector C está almacenado a continuación de B. Los siguientes registros están inicializados como se especifica a continuación:
 - PC=0x80000000
 - R6=0x80001000
 - R7=0x80001400
 - R8=0x80001800
 - R4 contiene i (inicialmente empieza en 0)
- Los vectores están inicializados como se especifica a continuación:
A[i]=i
B[i]=i*i
C[i]=0
- El sistema de memoria presenta las siguientes características:
 - Un único nivel de cache, separado en dos caches independientes, una para datos (**D\$**) y otra para instrucciones (**I\$**).
 - Las dos caches se encuentran inicialmente vacías.
 - La D\$ tiene la siguiente configuración:
 - Tamaño de 1KB
 - Tamaño de bloque de 16B
 - Acceso directo
 - Políticas de escritura: escritura directa (Write-Through) y sin asignación de escritura (No-Write-Allocate)
 - La I\$ tiene la siguiente configuración:
 - Tamaño de 512B
 - Tamaño de bloque de 16B
 - Acceso directo

Responde de forma razonada a las siguientes preguntas.

- a) (0.75 puntos) ¿Cuántos fallos de lectura tienen lugar en la D\$ y de qué tipo son? ¿Y cuántos de escritura?
- b) Calcula cómo afectaría a los fallos en la D\$ cambiar los siguientes aspectos software o hardware.
 1. (0.5) Alargar el array A en 4 elementos.
 2. (0.5 puntos) Utilizar una D\$ asociativa por conjuntos de dos vías.
 3. (0.5 puntos) Utilizar una D\$ asociativa por conjuntos de cuatro vías y con asignación de escritura (Write-Allocate).
- c) (0.75 puntos) ¿Cuántos fallos se producen en la I\$ y de qué tipo son (de inicio, conflicto o capacidad)? ¿Hasta qué valor podríamos reducir el tamaño de la I\$, manteniendo una política de acceso directo, sin afectar al número de fallos en la misma al ejecutar este código?

3.- (3 puntos) Sea un sistema de memoria virtual paginada de 8KB y una memoria física de 4 páginas de 512B cuyo algoritmo de reemplazamiento de páginas es LRU. En un momento dado, después de que el procesador haya generado las direcciones de memoria 0064, 0100, 1C1E, 1400, 049B, 03E0, 121E, 132F, 0A66, 08E5 (y estas direcciones hayan sido accedidas), el contenido de la memoria física es:

Nº Página Física	Nº Página Virtual
0	4
1	9
2	5
3	1

Se pide:

- a) (1 punto) Representar la tabla de páginas completa (con todas sus entradas) a partir de la información anterior, dejando en blanco los campos para los cuales no se han proporcionado los datos suficientes para ser completados. Para cada entrada de la tabla de páginas deben indicarse los campos: bit de válido, LRU (suponer un LRU igual a 0 para la última página accedida) y nº de página física.
- b) (0.75 puntos) Indicar el rango de direcciones virtuales y físicas de las páginas contenidas en la memoria física.
- c) (0.75 puntos) Mostrar cómo evoluciona el contenido de la memoria física y de la tabla de páginas si a continuación se referencian la siguiente cadena de direcciones virtuales: 1112, 1645, 0A49, 0622 y 17B4.
- d) (0.5 puntos) Si se añade una TLB asociativa de 2 entradas, ¿cuál sería su contenido tras el acceso a las direcciones virtuales del apartado anterior?